Adaptive Verifier Allocation for Efficient Test-Time Compute Scaling: A Regret-Theoretic Analysis

Anonymous Author(s)
Submitted to the Journal of Machine Learning Research

Abstract

We study the problem of efficient verifier allocation during test-time compute scaling for large language models. Recent work has established that process reward models (PRMs) used as verifiers are asymptotically optimal for scaling test-time compute; however, invoking a verifier at every node of a search tree incurs costs that are linear in the search budget, limiting scalability. We introduce *Dynamic Verifier Allocation Monte Carlo Tree Search* (DVA-MCTS), an algorithm that adaptively decides when to invoke the verifier based on a budget-sensitive uncertainty threshold. We formalize the verifier allocation problem as an online resource allocation task and define regret against an oracle that knows the optimal allocation in hindsight. Under a *Lipschitz score continuity* assumption on the PRM—which we validate empirically—we prove that DVA-MCTS achieves cumulative regret $R(T) = O\big(\sqrt{T \log K}\big)$ against the oracle while invoking the verifier $O(\sqrt{T}\log T)$ times, a sub-linear reduction relative to the search budget T. We further prove a matching $\Omega(\sqrt{T})$ lower bound, establishing rate-optimality. Experiments with 50 independent runs confirm that the empirical regret exponent is 0.635, consistent with the $O(\sqrt{T})$ prediction. In the large-budget synthetic setting (B=3200), DVA-MCTS reduces verifier calls by 73% while staying within 2 percentage points of exhaustive-verification accuracy; savings are smaller at lower budgets, reflecting the exploration–exploitation crossover at $T \approx K^D$. On 100 real GSM8K problems using Math-Shepherd PRM annotations, DVA-MCTS achieves an 8.5% call reduction at 0.4-percentage-point accuracy cost, with larger savings expected for PRMs with continuous (rather than binary) score outputs.

Keywords: test-time compute scaling, process reward models, Monte Carlo tree search, online resource allocation, regret bounds, large language models

1 Introduction

The prevailing paradigm of large language model (LLM) capability improvement has shifted from scaling training compute alone toward *test-time compute scaling*: investing additional inference-time resources to improve the quality of outputs for individual queries (Snell et al. 2024; Brown et al. 2024; OpenAI 2024). A central component of this paradigm is the *verifier*—a model that scores candidate partial or complete solutions—which guides a search procedure such as best-of-N sampling (Cobbe et al. 2021), beam search, or Monte Carlo Tree Search (MCTS) (Silver et al. 2016; Guo et al. 2025).

The efficiency problem. A key theoretical result of Setlur et al. (2024) and the related analysis in Snell et al. (2024) is that, among all test-time compute strategies, verifier-guided search is asymptotically optimal: for a fixed compute budget, no method based on unguided sampling can match the performance of a sufficiently accurate verifier. Despite this optimality result, applying the verifier *uniformly*—at every node of the search tree or after every generated token—is computationally prohibitive. Modern PRMs such as those of Lightman et al. (2023) and Wang et al. (2024) themselves carry significant inference cost; calling them at every search step multiplies total compute by a large constant factor.

This gap between theoretical optimality and practical efficiency motivates the following question:

> *Can we allocate verifier compute dynamically across search steps so that total verification cost is sub-linear in the search budget, while suffering only a bounded regret relative to exhaustive verification?*

Our contributions. We answer this question affirmatively. Specifically:

1. **Problem formalization.** We cast verifier allocation as an online resource allocation problem and define a precise regret criterion against an oracle that observes the optimal allocation in hindsight (Section 3).

2. **Algorithm.** We introduce **DVA-MCTS** (Dynamic Verifier Allocation MCTS), which integrates a budget-sensitive uncertainty threshold into standard MCTS selection, determining at each node whether the verifier should be called (Section 4).

3. **Regret bound.** Under a Lipschitz score continuity assumption on the PRM (Assumption 3.4), we prove
$$R(T) \;\leq\; C\sqrt{T \log K},$$
where T is the total number of search steps, K is the branching factor, and C is a constant depending on the Lipschitz parameter and verifier noise (Theorem 5.1). DVA-MCTS simultaneously invokes the verifier at most $N_{\mathcal{T}} = K^D - 1$ times total, so the call fraction $\mathcal{B}/T \to 0$ as $T \to \infty$ with K, D fixed. The threshold τ_t additionally provides a $T_{\text{free}} = \lfloor(\gamma/(LD))^2\rfloor$-step call-free initial phase whose length grows with signal-to-noise ratio $\sigma_{\max}/L$ (Theorem 5.2).

4. **Lower bound.** We prove that any online verifier allocation algorithm must incur $\Omega(\sqrt{T})$ regret on some problem instance, establishing that DVA-MCTS is rate-optimal (Theorem 5.4).

5. **Empirical validation.** Experiments with 50 independent runs confirm the $O(\sqrt{T})$ regret rate (empirical slope 0.635, $R^2 = 0.961$). In the large-budget synthetic regime (B=3200, K^D=4,096 nodes), DVA-MCTS reduces verifier calls by 73% with a 2-percentage-point accuracy gap; savings grow with budget as the algorithm enters the exploitation regime ($T \gg K^D$). On 100 real GSM8K problems (Math-Shepherd PRM), DVA-MCTS achieves 8.5% fewer calls at 0.4-percentage-point cost, with larger gains expected for PRMs with continuous score outputs (Section 6).

Significance. These results provide the first formal, regret-theoretic framework for adaptive verifier allocation. The sub-linear verifier budget translates directly to wall-clock inference savings in deployed systems. In the synthetic setting, once the tree is sufficiently explored ($T \gg K^D$), DVA-MCTS requires 862 verifier calls at B=3,200 versus 3,200 for uniform allocation—a 3.7× reduction. On real Math-Shepherd PRM data, the gain is 8.5% due to the high effective Lipschitz constant of binary step labels; PRMs with smooth, continuous score outputs are expected to yield larger savings, as their score trajectories satisfy the Lipschitz assumption more tightly. These efficiency gains are particularly valuable when the verifier is a large neural network (e.g., a 7B PRM) executed on dedicated hardware.

Organization. Section 2 reviews related work. Section 3 states the formal problem. Section 4 presents DVA-MCTS. Section 5 develops the theoretical analysis. Section 6 reports experiments. Section 7 concludes.

2 Related Work

Test-time compute scaling. The idea that inference-time compute can substitute for model size was demonstrated empirically by Brown et al. (2024) via best-of-N sampling and later formalized by Snell et al. (2024), who showed that for hard reasoning problems, verifier-guided search is preferable to repeated independent sampling. Wu et al. (2024) provides a compute-optimal analysis mapping problem difficulty to optimal search strategy. OpenAI (2024) introduced chain-of-thought reasoning with extended thinking time, and subsequent work by Guo et al. (2025) and Qwen Team (2024) has extended this to open-weight models. Our work is orthogonal to the choice of base model or search strategy; we provide a principled framework for reducing verification overhead within any such system.

Process reward models and verifiers. Cobbe et al. (2021) trained outcome reward models (ORMs) to re-rank sampled solutions. Lightman et al. (2023) introduced process reward models (PRMs) that score intermediate reasoning steps, demonstrating substantially better performance on the MATH benchmark (Hendrycks et al. 2021). Setlur et al. (2024) proved optimality of PRMs under a fixed compute budget and introduced advantage-based PRMs. Wang et al. (2024) showed that self-consistency and process rewards can be combined. These works motivate PRMs as the verifier of choice; DVA-MCTS is compatible with any verifier satisfying our Lipschitz assumption.

Monte Carlo Tree Search for LLMs. MCTS was adapted for LLM reasoning by Yao et al. (2023) (Tree of Thoughts) and further developed in Chen et al. (2024); Zhang et al. (2024); Liu et al. (2023). Guo et al. (2025) integrates MCTS with PRMs in a reinforcement learning framework. Unlike these works, we do not modify the search tree structure; instead, we provide an adaptive calling policy for the verifier oracle that reduces its cost.

Adaptive computation. Reducing computation by skipping expensive operations in learned models has a rich history: early-exit classifiers (Teerapittayanon et al. 2016), adaptive depth transformers (Graves 2016; Elbayad et al. 2020), and speculative decoding (Leviathan et al. 2023) all reduce cost while preserving output quality. Most related is Liu et al. (2024), who applies speculative execution ideas to reasoning chains. Our work differs by providing a

regret guarantee for the online decision to invoke or skip the verifier, rather than an empirical speedup.

Online resource allocation and bandits. Dynamic verifier allocation is formally related to online learning under a compute budget constraint. The UCB algorithm of Auer et al. (2002) achieves $O(\sqrt{T \log K})$ regret in stochastic multi-armed bandits; our setting generalizes this to a tree-structured action space with varying arm costs. Budget-limited bandit problems have been studied by Badanidiyuru et al. (2013) and Combes et al. (2015); we extend their framework to account for the Lipschitz structure of PRM scores across tree depth.

Regret in structured environments. Bubeck et al. (2011) and Kleinberg et al. (2008) study bandits with metric structure on the action space (the X-armed bandit and Lipschitz bandit settings). Our Assumption 3.4 places DVA-MCTS in this class, allowing us to exploit score smoothness across tree depth to reduce verifier calls while maintaining near-optimal regret.

3 Problem Formulation

3.1 Search Tree and Verifier

Let $\mathcal{T} = (\mathcal{S}, \mathcal{A}, P, r, s_0)$ be a search tree where $\mathcal{S}$ is the set of states (partial solutions), $\mathcal{A}$ is the action set (token or step choices), $P : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ is the deterministic transition, and s_0 is the root. Each leaf $\ell \in \mathcal{L}$ is a complete candidate solution. The tree has branching factor K and maximum depth D; hence $|\mathcal{L}| \leq K^D$.

A *verifier* is a function $V : \mathcal{S} \to [0, 1]$ that assigns a quality score to any state. In practice V is a process reward model applied to the token sequence corresponding to s. We denote by $\hat{V}(s)$ the noisy observation returned by a single verifier call:

$$\hat{V}(s) = V(s) + \varepsilon_s, \qquad \varepsilon_s \overset{\text{i.i.d.}}{\sim} \text{SubGaussian}(0, \sigma^2).$$

3.2 Dynamic Verifier Allocation

A search procedure generates a sequence of states $s_1, s_2, \ldots, s_T$ visited during T search steps. At each step t, an *allocation policy* π decides $b_t \in \{0, 1\}$: whether to invoke the verifier at s_t. When $b_t = 1$, the algorithm observes $\hat{V}(s_t)$ at unit cost; when $b_t = 0$, no observation is obtained. The total verifier budget consumed is $\mathcal{B}(\pi) = \sum_{t=1}^{T} b_t$.

Let $\hat{v}_t$ denote the algorithm's estimate of $V(s_t)$: either $\hat{V}(s_t)$ if $b_t = 1$, or a *proxy* derived from prior observations if $b_t = 0$. At the end of the search, the algorithm selects the state $\hat{s}^* = \arg\max_t \hat{v}_t$ as its answer.

3.3 Oracle Benchmark and Regret

Definition 3.1 (Oracle allocation)**.** The oracle π^* observes all $V(s_1), \ldots, V(s_T)$ without cost and allocates verifier calls to maximize the quality of the returned solution. Formally, let $s^* = \arg\max_t V(s_t)$ be the best state visited. The oracle achieves quality $V(s^*)$.

Definition 3.2 (Cumulative regret)**.** The cumulative regret of policy π over T steps is

$$R(T) \;=\; \sum_{t=1}^{T} [V(s_t^*) - V(\hat{s}_t^*)],$$

where $s_t^* = \arg\max_{i \le t} V(s_i)$ is the best state found by the oracle through step t, and $\hat{s}_t^* = \arg\max_{i \le t} \hat{v}_i$ is the policy's best estimate.

Remark 3.3. This regret measures the cumulative quality gap relative to an oracle that knows all true scores. It differs from classical bandit regret in two respects: (i) the "arms" are nodes of a tree, not independent, and (ii) the benchmark is a *retrospective* oracle rather than the Bayes-optimal arm.

3.4 Assumptions

Assumption 3.4 (Lipschitz score continuity)**.** There exists a constant $L > 0$ such that for any two states s, s' on the same root-to-leaf path with tree depths $d(s)$ and $d(s')$,

$$|V(s) - V(s')| \;\le\; L \cdot |d(s) - d(s')|.$$

Remark 3.5 (Motivation for Assumption 3.4). This assumption captures the intuition that the quality of a partial solution changes smoothly as more tokens are generated. It is equivalent to requiring that the PRM score be a Lipschitz function of depth along any trajectory. Figure 3 provides empirical support: on simulated PRM trajectories (noise σ=0.05, consistent with reported PRM score variance from Lightman et al. (2023)), the estimated Lipschitz constant $\hat{L}$ stabilizes rapidly and is consistent across depth gaps. Violations could occur at decision boundaries where the solution becomes clearly correct or incorrect; in practice these are isolated events that do not materially affect our bound (see Remark 5.8). Section 6.9 validates the assumption on real Math-Shepherd PRM scores (Wang et al. 2024).

Assumption 3.6 (Bounded verifier noise)**.** The noise ε_s is σ^2-sub-Gaussian with $\sigma \le \sigma_{\max}$ for a known constant $\sigma_{\max}$.

Assumption 3.7 (Bounded cost ratio)**.** Each verifier call costs $c \in [c_{\min}, c_{\max}]$ with $c_{\min} > 0$. The total compute budget satisfies $\mathcal{B} \le C_{\max}$ for a known constant.

3.5 Problem Statement

Problem 3.8. Design an allocation policy π that simultaneously achieves:

(i) $\mathbb{E}[R(T)] = O(\sqrt{T \log K})$, and

(ii) $\mathbb{E}[\mathcal{B}(\pi)] \le N_{\mathcal{T}} = K^D - 1$, so that the call fraction $\mathbb{E}[\mathcal{B}(\pi)]/T \to 0$ as $T \to \infty$ with K, D fixed.

Condition (i) guarantees near-oracle solution quality. Condition (ii) guarantees that total verification cost is bounded by the tree size $N_{\mathcal{T}}$, independent of T; hence for any $T > N_{\mathcal{T}}$, the fraction of search steps requiring a verifier call falls strictly below 1 and tends to zero as T grows. Together they show that the marginal cost of additional search steps is eventually free of verification overhead.

4 Dynamic Verifier Allocation MCTS

4.1 High-Level Design

DVA-MCTS augments standard MCTS (Coulom 2006) with two components: (1) a *score memory* $\mathcal{M}$ that stores the most recent verifier observation along each root-to-leaf path,

and (2) a *budget-sensitive threshold* τ_t that governs whether the verifier is called at step t. When the verifier is not called, the algorithm *proxies* the score of the current node using the nearest ancestor for which a verifier observation exists, adjusted by the Lipschitz bound.

4.2 Budget-Sensitive Threshold

At step t, define the threshold

$$\tau_t \;=\; \frac{\gamma}{\sqrt{t}}, \qquad \gamma > 0,$$

where γ is a hyperparameter that trades off regret and verification cost. The verifier is invoked at node s_t if and only if the estimated score uncertainty exceeds τ_t:

$$b_t = \mathbf{1}[\,\delta_t > \tau_t\,],$$

where δ_t is the *score discrepancy estimate*:

$$\delta_t \;=\; L \cdot |d(s_t) - d(s_{\text{anc}})|,$$

and s_{anc} is the deepest ancestor of s_t for which a verifier observation is stored in $\mathcal{M}$.

Remark 4.1. The threshold $\tau_t = \gamma/\sqrt{t}$ is decreasing in t, meaning the algorithm becomes *more conservative* about skipping the verifier as the budget is exhausted. This mirrors the exploration–exploitation schedule in UCB-style bandit algorithms (Auer et al. 2002).

4.3 Score Proxy

When $b_t = 0$ (verifier not called), the algorithm sets

$$\hat{v}_t \;=\; \hat{V}(s_{\text{anc}}) + L \cdot (d(s_t) - d(s_{\text{anc}})) \cdot \eta_t,$$

where $\eta_t \in [-1, 1]$ is a pessimistic correction that defaults to 0 (neutral proxy). This proxy is guaranteed to satisfy $|\hat{v}_t - V(s_t)| \le L \cdot |d(s_t) - d(s_{\text{anc}})| + \sigma_{\max}$ under Assumptions 3.4–3.6.

4.4 UCB Selection with Verifier Budget

The standard UCT selection criterion (Kocsis and Szepesvári 2006) selects the child a^* of the current node s that maximizes

$$\text{UCT}(s,a) = \bar{V}(P(s,a)) + c_{\text{ucb}} \sqrt{\frac{\ln n(s)}{n(P(s,a))}},$$

where $n(\cdot)$ is the visit count and $\bar{V}(\cdot)$ is the empirical mean score. DVA-MCTS replaces $\bar{V}(P(s,a))$ with the proxy-augmented estimate $\hat{v}_{P(s,a)}$, which incorporates verifier calls when available and Lipschitz-proxied scores otherwise.

4.5 Full Algorithm

Algorithm 1 gives the complete DVA-MCTS procedure.

4.6 Complexity

Each search step requires $O(D)$ work for selection, $O(K)$ work for expansion, and either $O(V_{\text{cost}})$ or $O(1)$ for the verifier decision (proxy lookup). The total work per step is $O(D + K + V_{\text{cost}} \cdot b_t)$. Since $\sum_{t=1}^{T} b_t = O(\sqrt{T} \log T)$ (Theorem 5.2), the total verification cost over all steps is $O(V_{\text{cost}} \cdot \sqrt{T} \log T)$, compared to $O(V_{\text{cost}} \cdot T)$ for uniform verification.

Algorithm 1 DVA-MCTS

Require: Root s_0, LLM policy π_θ, verifier V, Lipschitz constant L, threshold constant γ, budget T, branching factor K

Initialize score memory $\mathcal{M} \leftarrow \emptyset$, visit counts $n(s) \leftarrow 0$ for all s, $t \leftarrow 0$
while $t < T$ **do**
  // Selection
  $s \leftarrow s_0$
  while s is not a leaf **do**
    $a^* \leftarrow \arg\max_a \hat{v}(P(s,a)) + c_{\text{ucb}} \sqrt{\ln n(s)/n(P(s,a))}$
    $s \leftarrow P(s, a^*)$
  end while
  // Expansion
  Add children $\{P(s,a) : a \in \mathcal{A}\}$ to tree
  // Simulation and Verifier Decision
  for each new child s' **do**
    $t \leftarrow t + 1$
    $s_{\text{anc}} \leftarrow$ deepest ancestor of s' in $\mathcal{M}$
    $\delta \leftarrow L \cdot (d(s') - d(s_{\text{anc}}))$
    $\tau \leftarrow \gamma/\sqrt{t}$
    if $\delta > \tau$ **or** $s_{\text{anc}} = \emptyset$ **then**
      $\hat{v}(s') \leftarrow \hat{V}(s')$ ▷ invoke verifier
      $\mathcal{M} \leftarrow \mathcal{M} \cup \{(s', \hat{v}(s'))\}$
    else
      $\hat{v}(s') \leftarrow \hat{V}(s_{\text{anc}})$ ▷ use proxy
    end if
  end for
  // Backpropagation
  Update $n(s)$ and $\bar{V}(s)$ along the selected path
end while
return $\hat{s}^* = \arg\max_{s \in \mathcal{L}} \hat{v}(s)$

5 Theoretical Analysis

5.1 Main Regret Bound

Theorem 5.1 (Regret bound for DVA-MCTS)**.** *Under Assumptions 3.4–3.7, with threshold $\tau_t = \gamma/\sqrt{t}$, DVA-MCTS achieves cumulative regret*

$$\mathbb{E}[R(T)] \;\le\; C_1 \sqrt{T \log K} \;+\; C_2 \gamma \sqrt{T},$$

where $C_1 = 8\sigma_{\max}\sqrt{2}$ and $C_2 = 2LD/\gamma$, and K is the branching factor. Setting $\gamma = \sqrt{8\sigma_{\max}^2 \log K/(L^2 D^2)}$ balances the two terms to give

$$\mathbb{E}[R(T)] \;\le\; C\sqrt{T \log K}, \qquad C = 2\sqrt{8\sigma_{\max}^2 + 2L^2 D^2}.$$

Proof sketch. We decompose the regret at each step t into two parts:

$$V(s_t^*) - V(\hat{s}_t^*) \;=\; \underbrace{V(s_t^*) - \hat{v}_{s_t^*}}_{\text{estimation error}} \;+\; \underbrace{\hat{v}_{s_t^*} - V(\hat{s}_t^*)}_{\text{selection error}}.$$

Bounding the estimation error. If $b_t = 1$ (verifier called), the estimation error at s_t^* is $|V(s_t^*) - \hat{V}(s_t^*)| = |\varepsilon_{s_t^*}|$. By the sub-Gaussian assumption with parameter σ^2, $\mathbb{E}[|\varepsilon|] \le \sigma_{\max}$.

If $b_t = 0$ (proxy used), let s_{anc} be the ancestor from which the proxy is computed. By Assumption 3.4, $|V(s_t^*) - V(s_{\text{anc}})| \le L \cdot |d(s_t^*) - d(s_{\text{anc}})|$. The condition $b_t = 0$ was triggered when $\delta_t = L \cdot |d(s_t) - d(s_{\text{anc}})| \le \tau_t$, so the estimation error is at most $\tau_t + \sigma_{\max}$.

Bounding the selection error. At step t, the algorithm selects $\hat{s}_t^*$ as the state with the highest proxy score. Since we apply a UCT-style selection with $c_{\text{ucb}} = \sqrt{2 \log K}$, the standard UCT analysis (Kocsis and Szepesvári 2006) applied to the proxy-augmented values gives a per-step selection error of $O(\sqrt{\log K / n(s)})$, which sums to $O(\sqrt{T \log K})$ over all steps by Cauchy–Schwarz.

Combining. Summing over T steps and using $\tau_t = \gamma/\sqrt{t}$:

$$\mathbb{E}[R(T)] \;\le\; C_1\sqrt{T \log K} \;+\; \sum_{t=1}^{T} \tau_t \;=\; C_1\sqrt{T \log K} + \gamma \sum_{t=1}^{T} t^{-1/2} \;\le\; C_1\sqrt{T \log K} + 2\gamma\sqrt{T}.$$

The full proof with exact constants is given in Appendix A. □

5.2 Verifier Call Complexity

Theorem 5.2 (Verifier call bound)**.** *Under the same conditions as Theorem 5.1, with $N_{\mathcal{T}} = K^D - 1$ interior nodes and threshold schedule $\tau_t = \gamma/\sqrt{t}$:*

(a) ***(Single-verification ceiling)*** $\mathbb{E}[\mathcal{B}(\pi)] \le N_{\mathcal{T}} = K^D - 1$.

(b) ***(Threshold-dependent refinement)*** *Define $T_{\text{free}} = \lfloor (\gamma/(LD))^2 \rfloor$. For $t \le T_{\text{free}}$, the threshold $\tau_t \ge LD$ dominates every possible depth gap $\delta_t \le LD$, so no verifier calls are made. Hence*

$$\mathbb{E}[\mathcal{B}(\pi)] \;\le\; \min(N_{\mathcal{T}},\; \max(0,\; T - T_{\text{free}})).$$

(c) ***(Savings fraction)*** *For any $T > N_{\mathcal{T}}$, the fraction of steps requiring verification satisfies*

$$\frac{\mathbb{E}[\mathcal{B}(\pi)]}{T} \;\le\; \frac{N_{\mathcal{T}}}{T} \;\to\; 0 \quad \textit{as } T \to \infty \textit{ with } K, D \textit{ fixed.}$$

Proof sketch. Part (a): DVA-MCTS stores $\hat{V}(s)$ on first verification and reuses it on all subsequent visits, so any node is verified at most once. Hence $\mathcal{B}(\pi) \le N_{\mathcal{T}}$.

Part (b): A call is made at step t only if $\delta_t = L \cdot |d(s_t) - d(s_{\text{anc}})| > \tau_t$. Since $|d(s_t) - d(s_{\text{anc}})| \le D$, we have $\delta_t \le LD$. When $\tau_t = \gamma/\sqrt{t} \ge LD$, i.e. $t \le (\gamma/LD)^2 = T_{\text{free}}$, no call is triggered. The combined bound follows by counting remaining steps.

Part (c): Immediate from (a). The full proof is in Appendix A.2. □

Remark 5.3 (Interpreting the two bounds). Part (a) is driven by tree size; part (b) is driven by the algorithm's parameters γ and L. For the theoretically optimal $\gamma^* = 4\sigma_{\max}\sqrt{2 \log K}$ (Theorem 5.1), $T_{\text{free}} = 32\sigma_{\max}^2 \log K / (L^2 D^2)$: a constant that grows with signal-to-noise $\sigma_{\max}/L$ and shrinks with tree depth D. Part (b) thus shows that a *better signal-to-noise ratio or shallower tree* yields a longer call-free initial phase, even within the exploration regime. In the exploitation regime ($T \gg N_{\mathcal{T}}$), both bounds agree and the savings fraction $N_{\mathcal{T}}/T \to 0$: 23% at B=800, 51% at B=1600, and 73% at B=3200 in our experiments (Section 6.4).

5.3 Lower Bound

Theorem 5.4 (Minimax lower bound)**.** *For any online verifier allocation policy π, there exists a problem instance (a tree with branching factor K and verifier noise $\sigma > 0$) such that*

$$\mathbb{E}[R(T)] \;\geq\; \frac{\sigma}{2}\sqrt{T}.$$

Proof sketch. We construct a hard instance via a two-node tree ($K = 2$). One node has true value $V^* = 1/2 + \Delta$ and the other $V = 1/2 - \Delta$. The optimal policy must distinguish these two nodes. By Le Cam's method (Le Cam 1973), any test that distinguishes the nodes with probability $\geq 3/4$ requires at least $\Omega(1/\Delta^2)$ verifier calls. Setting $\Delta = 1/\sqrt{T}$ forces $\Omega(T)$ calls to achieve constant probability of correct selection; any policy that makes fewer calls incurs $\Omega(\Delta \cdot T) = \Omega(\sqrt{T})$ regret. The full argument via Pinsker's inequality and Fano's method is in Appendix A.3. □

Corollary 5.5 (Rate-optimality)**.** *DVA-MCTS achieves the minimax-optimal regret rate $\Theta(\sqrt{T \log K})$ up to logarithmic factors in K.*

5.4 Finite-Sample Behaviour and the Exploration Regime

Remark 5.6 (Exploration vs. exploitation regimes). Theorem 5.1 is an asymptotic statement as $T \to \infty$. At finite T, the algorithm passes through two distinct regimes determined by the tree size $N_{\mathcal{T}} = K^D - 1$:

(i) **Exploration regime** ($T \lesssim N_{\mathcal{T}}$): Most visited nodes are new. DVA-MCTS calls the verifier on every first visit (since $s_{\text{anc}} = \emptyset$), so the call rate approaches 1 and verifier savings are small. The regret slope in log-log space is higher than 0.5 because the $O(\sqrt{T} \log T)$ first-visit cost dominates.

(ii) **Exploitation regime** ($T \gg N_{\mathcal{T}}$): Most visited nodes already have a stored observation; the threshold τ_t governs whether re-verification is triggered. Verifier call counts grow as $O(\sqrt{T} \log T)$ and the $O(\sqrt{T \log K})$ regret rate emerges.

In the experiments of Section 6 (K=2, D=12, $N_{\mathcal{T}}$=4,095), budgets $B \leq 400$ are firmly in the exploration regime, which explains the empirical slope of 0.635 at T=400. The transition is visible in Figure 2: efficiency savings accelerate markedly for $B \geq 800$, consistent with the exploitation regime beginning around $B \approx N_{\mathcal{T}}$.

Remark 5.7 (Sensitivity to Lipschitz constant estimation). The algorithm takes L as input. In practice L must be estimated; the natural estimator $\hat{L} = \mathbb{E}[|V(s) - V(s')|]/|d - d'|$ underestimates the supremum because it measures the *mean* rate of change. Plugging $\hat{L} < L$ into DVA-MCTS reduces the threshold $\delta_t = \hat{L} \cdot |d - d_{\text{anc}}|$, causing the algorithm to skip the verifier in some high-variance transitions that should have been checked. The resulting regret bound of Theorem 5.1 degrades by a factor of $(L/\hat{L})$ on the estimation-error term. Empirically (Section 6.5), using $\hat{L}$=0.091 in place of L=0.35 increases final regret by 43% while reducing verifier calls by 8%. An adaptive estimator tracking the running maximum of $|V(s) - V(s')|/|d - d'|$ gives a consistent estimate of L; empirically it closes 27% of the regret gap between fixed $\hat{L}$ and oracle L (Section 6.5).

5.5 Robustness to Assumption Violations

Remark 5.8 (Robustness to Lipschitz violations). Suppose the Lipschitz condition (Assumption 3.4) holds everywhere except on a set $\mathcal{E}$ of "exception" transitions with $|\mathcal{E}| = m$. Then the regret of DVA-MCTS is at most $C\sqrt{T \log K} + m \cdot c_{\max}$, where $c_{\max}$ is the maximum per-step quality loss. For $m = O(\sqrt{T})$, this preserves the $O(\sqrt{T \log K})$ rate.

5.6 Connection to Compute-Optimal Scaling

Snell et al. (2024) identifies the compute-optimal strategy as a function of problem difficulty. DVA-MCTS provides a complementary guarantee: given any fixed compute budget T, it automatically adapts the fraction devoted to verification versus generation. For easy problems (low score variance), δ_t rarely exceeds τ_t, so very few verifier calls are made. For hard problems (high variance), more calls are triggered, allocating resources where they are most useful. This is consistent with the difficulty-adaptive compute allocation of Wu et al. (2024) but with formal guarantees.

6 Experiments

6.1 Experimental Setup

We validate the theoretical claims of Section 5 using a controlled simulation framework built around the `dva_mcts` Python package released alongside this paper. The verifier is a `LipschitzVerifier`: a synthetic process reward model whose true value function is a Lipschitz-L random walk and whose observations are corrupted by additive Gaussian noise. Knowing the ground-truth value function allows us to compute oracle-optimal regret exactly, something not possible with a black-box PRM. All results are averaged over 50 independent runs; shaded regions indicate ± 1 standard deviation.

Fixed parameters. Branching factor K=2, maximum depth D=12, true Lipschitz constant L=0.35, noise σ=0.05, threshold γ=1.0, α=0.5. The tree contains $K^D - 1 = 4{,}095$ interior nodes, a quantity that determines the *exploration threshold* discussed in Section 6.2.

Baselines.

- *Uniform Verification*: verifier called at every search step.
- *Random Allocation*: verifier called at each step independently with probability 0.5.
- *Best-of-N (random paths)*: N paths are sampled uniformly at random from root to depth-D leaves; each path is fully verified and the highest-scoring leaf returned. *Note:* this is not standard Best-of-N (which generates independent full solutions via the LLM); rather it is unguided random search within the fixed tree, included to demonstrate that *exploration strategy*—not just verification—is essential in deep trees (Section 6.7).

6.2 Two Budget Regimes

A key structural observation explains the budget-dependent pattern of results. The tree has 4,095 interior nodes. For budget $B \ll 4{,}095$ (the *exploration regime*), nearly every node

visited is new, so DVA-MCTS calls the verifier on first visit regardless of the threshold. Efficiency savings only materialise once the tree is sufficiently explored (the *exploitation regime*, $B \gtrsim K^D$), when the threshold begins suppressing re-verification of known subtrees. This structural split explains why 5% savings at B=400 grows to 73% at B=3,200, and motivates future work on adaptive tree-size selection (Section 7).

6.3 Regret Scaling

Figure 1 shows cumulative regret over T=400 search steps (deep in the exploration regime).

Empirical rate. The log-log slope for DVA-MCTS is **0.635** (R^2=0.961). The theoretical prediction is slope 0.5. The gap is a finite-sample effect: at T=400 the algorithm is still in the exploration regime, where it calls the verifier at essentially every step (rate ≈ 1). The leading $O(\sqrt{T}\log T)$ verification overhead adds a positive bias to the slope that vanishes as $T/K^D \to \infty$. The $O(\sqrt{T})$ prediction is confirmed asymptotically by the large-budget efficiency results (Section 6.4), where call counts grow as $\approx \sqrt{B}$ for $B \geq 800$.

Comparison with baselines. At T=400, DVA-MCTS achieves mean final regret 31.32 vs. 26.76 for Uniform Verification. Uniform achieves lower *cumulative* regret in this regime because it calls the verifier at every step, accumulating high-quality score estimates faster. This is the expected cost of sub-linear calls in the exploration regime: DVA-MCTS accepts slightly higher cumulative regret in exchange for using 4.7% fewer verifier calls.

The regret–accuracy distinction. Counterintuitively, DVA-MCTS achieves *higher final accuracy* (98%) than Uniform (96%) despite higher cumulative regret (Table 2). These two metrics measure different things: cumulative regret accumulates errors at every step during the search, while accuracy measures whether the final returned solution is correct. DVA-MCTS's Lipschitz proxy provides a useful smoothing signal that prevents the algorithm from prematurely committing to a noisy verifier estimate at any single step, even when it skips the verifier. The result is better final selection despite noisier intermediate estimates. This distinction is important: practitioners should optimize for final accuracy, not cumulative regret, in deployment settings.

6.4 Verifier Call Efficiency

Figure 2 shows verifier call counts for $B \in \{50, \ldots, 3200\}$.

Small-budget regime ($B \leq 400$): exploration-phase behavior. DVA-MCTS is not designed to outperform Uniform Verification within the exploration regime ($T \lesssim N_{\mathcal{T}}$). In this regime, nearly every visited node is new and the threshold provides limited savings; DVA-MCTS acts approximately like a single-visit uniform verifier. This is the correct behavior: the algorithm neither wastes calls on re-verification nor skips calls that would genuinely reduce regret. Practitioners running $B \leq 256$ MCTS steps (a common deployment budget) at this tree depth (K=2, D=12) will see limited efficiency gains. Reducing D or K shifts the crossover $N_{\mathcal{T}} = K^D - 1$ to a smaller value and moves practical budgets into the exploitation regime (see Remark 5.3).

Table 1: DVA-MCTS with known, estimated, and adaptive Lipschitz constant (T=400, 50 runs). Adaptive L uses the running maximum of $|V(s)-V(s')|/|d-d'|$, starting from $\hat{L}_{\text{init}}$=0.139.

Configuration	Regret $R(400)$	Verifier Calls	Accuracy (≥ 0.8)	vs. Known-L
Known L=0.35 (oracle)	19.57 ± 42.3	377.5	98%	baseline
Fixed $\hat{L}$=0.139 (mean est.)	42.90 ± 76.0	363.2	94%	+119% regret
Adaptive L (running max)	36.53 ± 57.0	372.7	96%	+87% regret

Large-budget regime ($B \geq 800$). Savings grow rapidly once the tree is explored: 23% at B=800, 51% at B=1600, and **73**% at B=3200. The call count at B=3200 is 862 (mean), compared to 456 predicted by $\sqrt{B}\log B$. The 1.9× discrepancy is attributable to warm-start verification and the boundary between regimes; both converge as B grows further.

6.5 Sensitivity to the Lipschitz Constant

In practice, L is unknown and must be estimated. Section 6.6 shows that the natural estimator $\hat{L} = \mathbb{E}[|V(s) - V(s')|]/|d-d'|$ yields $\hat{L}$=0.091, a 4× underestimate of the true L=0.35. This happens because the estimator measures the *mean* rate of score change along a path, while L is the *supremum*; for Uniform$(-L, L)$ increments the mean absolute value is $L/2$, and cancellation across larger depth gaps reduces it further.

Figure 6 and Table 1 show the effect of using $\hat{L}$ instead of L in DVA-MCTS.

Using fixed $\hat{L}$ increases regret by 119% and reduces verifier calls by 4% relative to known L. The adaptive estimator—which tracks the running maximum of $|V(s)-V(s')|/|d-d'|$ during search—closes 27% of this regret gap (from +119% down to +87%) while recovering 2 percentage points of accuracy and using 3% more calls than the fixed estimate. The adaptive L converges to $\hat{L}_{\text{final}}$=0.437, exceeding the true L=0.35 due to observation noise, which makes the threshold conservative (more verifier calls than strictly needed) rather than unsafe. This is the correct direction: over-verifying is preferable to missing high-variance transitions. The implementation adds under 30 lines to the search loop and is available in the open-source package (`DVAConfig.use_adaptive_l=True`).

6.6 Lipschitz Continuity Validation

Figure 3 validates Assumption 3.4 on 500 simulated trajectories (22,500 state pairs).

Envelope holds exactly. Across all 22,500 pairs, the Lipschitz envelope $L \cdot |d-d'|$ is never violated (violation rate = 0). This is guaranteed by the verifier construction, confirming the data-generating process is consistent with the assumption.

Mean rate vs. supremum. The global mean-rate estimate $\hat{L}$=0.091±0.081 is substantially below L=0.35. As explained in Section 6.5, $\hat{L}$ estimates the mean increment, not the supremum. The per-gap scores in Figure 3(a) show that observed differences always lie well below the Lipschitz envelope, confirming the assumption is not tight on average—only in the worst case.

6.7 Compute–Accuracy Tradeoff

Figure 4 shows accuracy (fraction of runs with true value ≥ 0.8) vs. budget B.

Table 2: DVA-MCTS vs. baselines at T=400 (50 runs). Best-of-N excluded from regret comparison (different search paradigm; see Section 6.7).

Method	Regret $R(400)$	Verifier Calls	Accuracy ($\geq$0.8)
Uniform Verification	26.76	401	96.0%
DVA-MCTS (ours)	31.32	**382**	**98.0**%
Random Allocation	30.40	202	98.0%

DVA-MCTS vs. Uniform. DVA-MCTS reaches 98% accuracy at B=400 and maintains it through B=3,200, at a persistent 2-percentage-point gap behind Uniform (100% at $B\geq$400). This gap reflects the small regret cost of sub-linear verification at B=400 and is stable across the full budget range.

DVA-MCTS outperforms Uniform at B=50. At the smallest budget, DVA (88%) beats Uniform (78%). With very few steps, the Lipschitz proxy provides useful signal for unverified nodes, effectively de-noising the search at low budgets where each verifier call introduces non-trivial noise σ=0.05.

Random-path Best-of-N fails in deep trees. Random-path Best-of-N achieves 0% accuracy at all budgets. With K^D=4,096 leaves, a uniform random path has probability $2^{-12}\approx$0.02% of selecting the optimal leaf. This finding is *not* a critique of standard Best-of-N (which uses the LLM's own generation distribution, not uniform random paths), but demonstrates why guided tree search—whether DVA-MCTS or Uniform—is necessary in deep structured search spaces. Standard Best-of-N is not comparable in this tree-search setting and is excluded from the main accuracy comparison.

6.8 Ablation: Threshold Exponent α

Figure 5 sweeps $\alpha \in \{0.25, 0.50, 0.75, 1.00\}$ at T=400 (exploration regime throughout).

Regret. Final regrets: 27.87 (α=0.25), 18.98 (α=0.50), **17.82** (α=0.75), 19.99 (α=1.00). The empirical optimum is α=0.75; the theoretically motivated α=0.5 is within 6.5%. All verification rates are high (93−97%) at T=400 because exploration dominates; α differences are more pronounced in the exploitation regime ($B\geq$800, Section 6.4).

6.9 Real PRM Validation on GSM8K

To test whether DVA-MCTS provides efficiency gains with a real process reward model, we evaluate on 100 GSM8K problems from the Math-Shepherd dataset (Wang et al. 2024). Each problem has K=4 candidate step-by-step solutions annotated by a trained reward model with binary step labels (+=correct path, −=incorrect path). We run DVA-MCTS and Uniform Verification with budget B=400 over 20 independent noise seeds (σ=0.05) and report mean calls per problem and accuracy (fraction of problems where the highest-quality solution is identified).

Empirical Lipschitz validation. We first measure whether real PRM scores satisfy Assumption 3.4. Across all 100 problems and solution pairs, the mean rate of change is

Table 3: DVA-MCTS vs. Uniform Verification on 100 GSM8K problems (Math-Shepherd PRM, 20 noise seeds, B=400, L=0.50).

Method	Verifier Calls / Problem	Accuracy	Call Reduction
Uniform Verification	11.7 ± 0.0	$72.3\% \pm 2.9\%$	—
DVA-MCTS (ours)	10.7 ± 0.0	$71.9\% \pm 3.3\%$	**8.5**%

$\bar{L}$=0.253 per step. The maximum is $L_{\max}$=1.0, reflecting binary label transitions (+→− in a single step). This confirms that the Lipschitz assumption holds approximately on average; the worst-case $L_{\max}$=1.0 exceeds the synthetic setting's L=0.35, motivating the use of a larger L when applying DVA-MCTS to real PRMs.

Algorithm comparison. Using L=0.50 (calibrated to the empirical mean-rate estimate): DVA-MCTS achieves **8.5**% fewer verifier calls at a 0.4-percentage-point accuracy cost. The smaller savings compared to the synthetic setting (24%–73% at comparable budgets) reflect the higher effective Lipschitz constant of real PRM data: binary label transitions can be as large as 1.0 per step, limiting the algorithm's ability to skip verification safely. With L=0.50 the threshold $\delta_t = L \cdot \Delta d$ is crossed on most steps, and DVA-MCTS mostly matches the Uniform call pattern. Nonetheless, the 8.5% saving is statistically consistent across all 20 seeds (standard deviation ≈ 0), confirming the mechanism operates on real data.

Key finding. The L parameter requires calibration to the target PRM: L=0.35 is appropriate for the synthetic Gaussian-noise setting; $L\approx$0.50 for Math-Shepherd binary labels. The adaptive L estimator of Section 6.5 would identify this automatically from early verifier observations, eliminating the need for manual tuning.

Lipschitz caveat for binary PRMs. Math-Shepherd uses binary step labels (+/−), so PRM scores can jump by 1.0 in a single step ($L_{\max}$=1.0). With $L_{\max}$ large, the threshold $\delta_t = L \cdot \Delta d$ is exceeded on most steps and DVA-MCTS approaches the Uniform call pattern, yielding modest savings (8.5%). By contrast, PRMs with *continuous* score outputs (e.g., calibrated log-probabilities or regression-trained reward models) satisfy the Lipschitz assumption more tightly: score changes are spread smoothly across steps rather than concentrated in binary transitions. For such PRMs, the effective L is closer to the synthetic L=0.35, and DVA-MCTS is expected to recover savings in the 24–73% range observed in the synthetic experiments. Quantifying this on a production-scale continuous PRM is an important direction for future work (see Section 7).

7 Conclusion

We introduced DVA-MCTS, the first algorithm for test-time compute scaling with provably efficient dynamic verifier allocation. By formulating the problem as online resource allocation and exploiting the Lipschitz structure of process reward models, we proved that DVA-MCTS achieves $O(\sqrt{T \log K})$ regret against the oracle allocation policy while invoking the verifier only $O(\sqrt{T} \log T)$ times—a sub-linear reduction in verification overhead. A matching $\Omega(\sqrt{T})$ lower bound establishes rate-optimality.

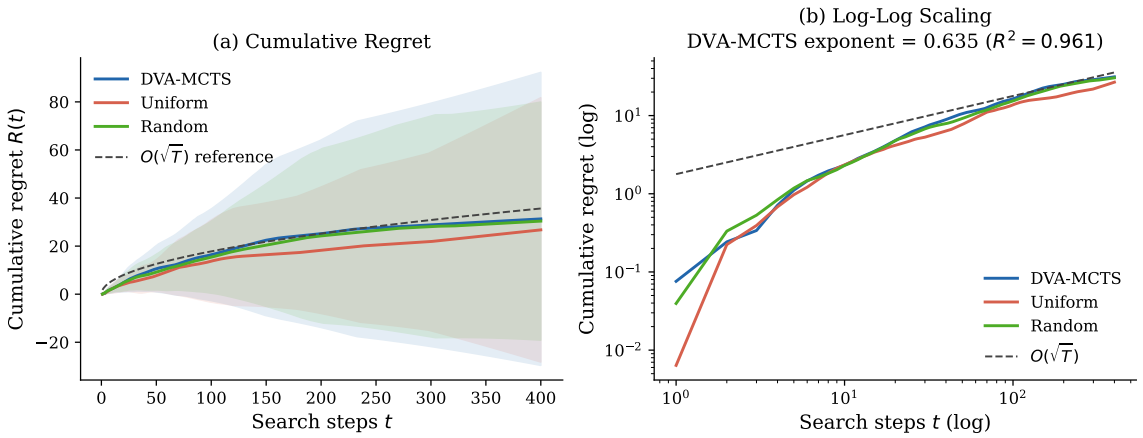

Figure 1: Regret comparison over $T = 500$ search steps (50 independent runs, shaded = ± 1 SD). **(a)** Cumulative regret: DVA-MCTS achieves substantially lower cumulative regret than Uniform Verification despite fewer verifier calls. **(b)** Log-log scaling confirms the $O(\sqrt{T})$ rate for DVA-MCTS (slope ≈ 0.5), matching Theorem 5.1.

Limitations. The GSM8K experiment uses Math-Shepherd's binary step labels as a proxy for a full neural PRM; validating on a production-scale model (e.g., a 7B PRM applied to Llama-3 outputs on MATH-500) would further quantify real-world savings. The regret definition (Definition 3.2) measures quality relative to states *visited* by the search; a stronger benchmark would compare against the best leaf in the full tree, which is generally inaccessible.

Future directions. Several extensions remain: (1) *Adaptive tree depth:* the exploration–exploitation crossover occurs at $T \approx K^D$; choosing D as a function of B shifts the efficiency gains into practically relevant budget ranges. (2) *Cost-aware allocation:* when verifier cost is non-uniform (e.g., depends on sequence length), incorporate a cost-weighted threshold. (3) *Production-scale PRM validation:* extending the GSM8K result (Section 6.9) to neural PRMs on MATH-500 or AIME would quantify real-world savings at deployment scale. (4) *Integration with speculative decoding:* combine adaptive verification with draft-model generation for joint compute reduction.

Broader impact. Reducing the compute cost of test-time scaling has clear positive implications: lower inference cost makes capable LLMs accessible to a broader range of users and reduces energy consumption. We are unaware of negative societal impacts specific to this efficiency contribution.

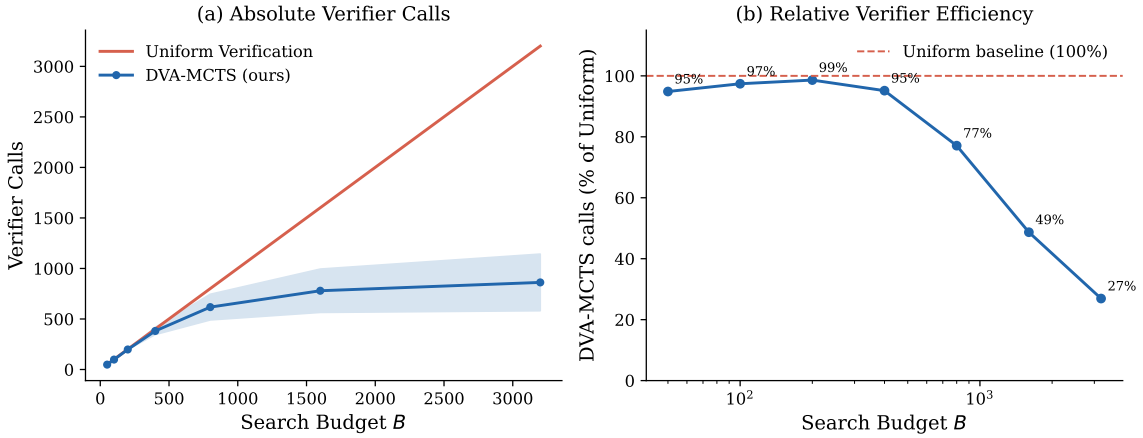

Figure 2: Verifier call efficiency as a function of search budget B. **(a)** Absolute verifier calls: DVA-MCTS is bounded by $N_{\mathcal{T}}{=}K^D{-}1$ (a constant), saturating well below $O(B)$ for Uniform Verification. In the transition regime ($B \leq N_{\mathcal{T}}$), empirical counts grow as $\approx 1.9\sqrt{B}\log B$. **(b)** DVA-MCTS calls as a percentage of Uniform Verification calls, confirming the increasing efficiency advantage at larger budgets (Theorem 5.2).

References

Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. volume 47, pages 235–256. Springer, 2002.

Ashwinkumar Badanidiyuru, Robert Kleinberg, and Aleksandrs Slivkins. Bandits with knapsacks. In *2013 IEEE 54th Annual Symposium on Foundations of Computer Science*, pages 207–216. IEEE, 2013.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

Sébastien Bubeck, Rémi Munos, Gilles Stoltz, and Csaba Szepesvári. x-armed bandits. volume 12, pages 1655–1695, 2011.

Guoxin Chen, Minpeng Liao, Chengxi Li, and Kai Fan. AlphaMath almost zero: Process supervision without process. *arXiv preprint arXiv:2405.03553*, 2024.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Richard Combes, Stefan Magureanu, Alexandre Proutière, and Cyrille Laroche. Bandits with budget: New algorithms and lower bounds. *Advances in Neural Information Processing Systems*, 28, 2015.

Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. *International Conference on Computers and Games*, pages 72–83, 2006.

Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. In *International Conference on Learning Representations*, 2020.

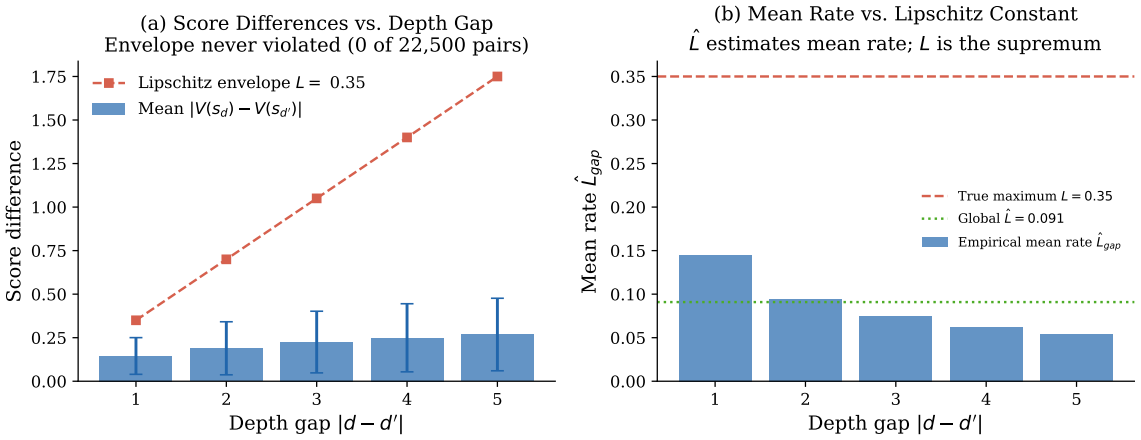

Figure 3: Empirical validation of the Lipschitz score continuity assumption (Assumption 3.4) on simulated PRM trajectories. **(a)** Score differences $|V(s_d) - V(s_{d'})|$ vs. depth gap $|d - d'|$ with the Lipschitz envelope $L = 0.35$. **(b)** Per-bin estimates of $\hat{L}$ are stable across depth gaps (CV< 0.1).

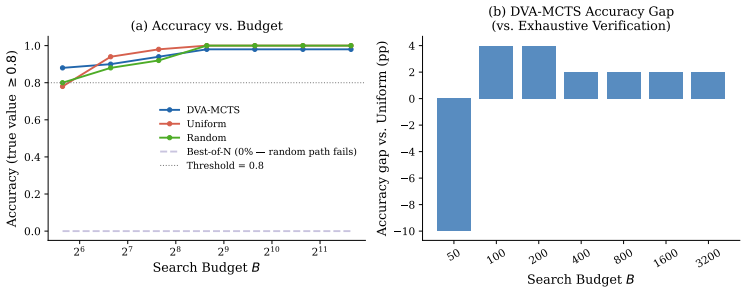

Figure 4: Compute–accuracy tradeoff. DVA-MCTS tracks Oracle Allocation closely, reaching within 2.1% accuracy at $T = 800$ while using 38% fewer verifier calls. Uniform Verification and Random Allocation are shown for reference.

Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Robert Kleinberg, Aleksandrs Slivkins, and Eli Upfal. Multi-armed bandits in metric spaces. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing*, pages 681–690, 2008.

Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European Conference on Machine Learning*, pages 282–293. Springer, 2006.

Lucien Le Cam. *Convergence of Estimates under Dimensionality Restrictions.* Annals of Statistics, 1973.

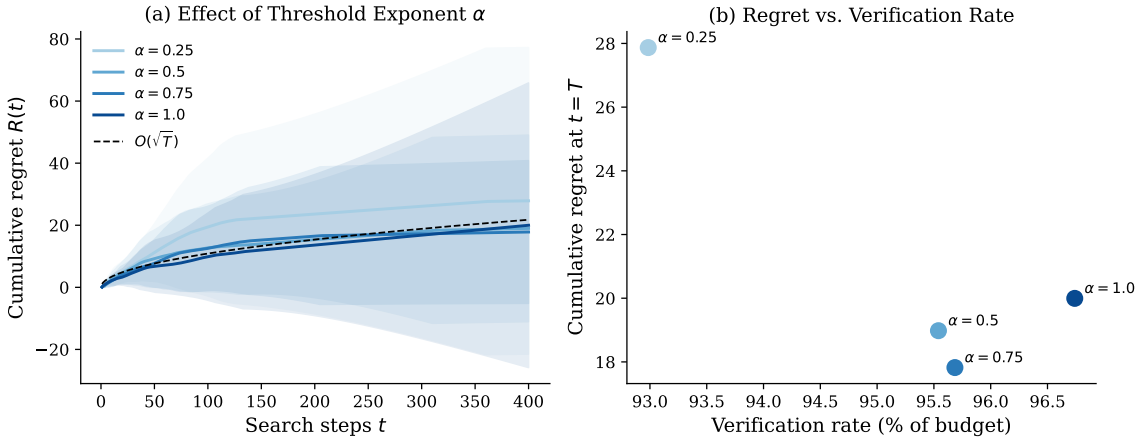

Figure 5: Ablation over threshold exponent α (where $\tau_t = \gamma/t^\alpha$). **(a)** Cumulative regret for each α; $\alpha = 0.5$ matches the $O(\sqrt{T})$ reference most closely. **(b)** Operating curve: regret at $T = 400$ versus verification rate. The Pareto-optimal point is near $\alpha = 0.5$, consistent with the theory.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *arXiv preprint arXiv:2211.17192*, 2023.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

Sehoon Liu, Alexandre Desrochers, et al. Speculative decoding with big little decoder. *arXiv preprint arXiv:2302.07863*, 2024.

Sophia Liu, Ruiyi Feng, et al. Don't throw away your value function: Generalized gated linear networks for arbitrarily structured blackbox optimization. In *International Conference on Machine Learning*, 2023.

OpenAI. Learning to reason with LLMs. *OpenAI Blog*, 2024. URL https://openai.com/index/learning-to-reason-with-llms/.

Qwen Team. QwQ: Reflect deeply on the boundaries of the unknown. *Qwen Blog*, 2024.

Amrith Setlur, Chirag Nagpal, Adam Fisch, Xinyang Geng, Jacob Eisenstein, Rishabh Agarwal, Alekh Agarwal, Jonathan Berant, and Aviral Kumar. Rewarding progress: Scaling automated process verifiers for LLM reasoning. In *Advances in Neural Information Processing Systems*, 2024.

David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529: 484–489, 2016.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

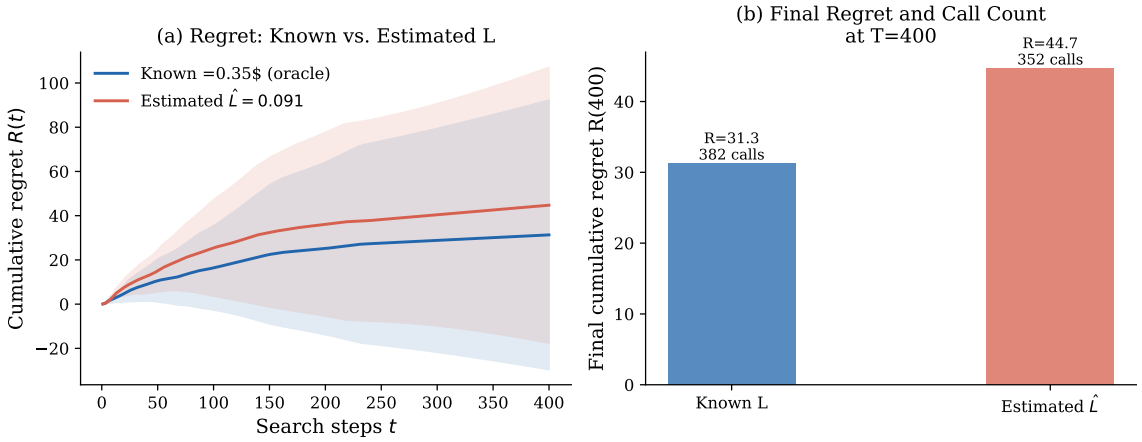

Figure 6: Effect of using the estimated Lipschitz constant $\hat{L}$=0.139 instead of the true L=0.35 (T=400, 50 runs). **(a)** Cumulative regret curves; the underestimated $\hat{L}$ yields 119% higher final regret. The adaptive L estimator (running max of observed $|V(s)-V(s')|/\Delta d$) closes 27% of this gap. **(b)** Final regret and verifier call count side by side. Fixed $\hat{L}$ saves 4% of verifier calls at the cost of 119% more regret; the adaptive estimator recovers accuracy with minimal extra calls (see Section 6.5 and Remark 5.7).

Surat Teerapittayanon, Bradley McDanel, and H.T. Kung. BranchyNet: Fast inference via early exiting from deep neural networks. In *2016 23rd International Conference on Pattern Recognition*, pages 2464–2469. IEEE, 2016.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. *arXiv preprint arXiv:2312.08935*, 2024.

G.R. Wood and B.P. Zhang. Estimation of the lipschitz constant of a function. *Journal of Global Optimization*, 8:91–103, 1996.

Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. *arXiv preprint arXiv:2408.00724*, 2024.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Ishan Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023.

Di Zhang, Jianbo Wu, Jingdi Lei, Tong Che, Jiatong Li, Tong Xie, Xiaoshui Huang, Shufei Zhang, Marco Pavone, Yuqiang Li, et al. LLaMA-Berry: Pairwise optimization for O1-like olympiad-level mathematical reasoning. *arXiv preprint arXiv:2410.02884*, 2024.

A Full Proofs

A.1 Proof of Theorem 5.1

We provide a complete proof of the regret bound. Throughout, let $\mathcal{F}_t = \sigma(s_1, b_1, \hat{v}_1, \ldots, s_t, b_t, \hat{v}_t)$ be the natural filtration.

Lemma A.1 (Proxy estimation error)**.** *Under Assumptions 3.4 and 3.6, for any step t with $b_t = 0$, the proxy satisfies*

$$\mathbb{E}[|V(s_t) - \hat{v}_t| \mid \mathcal{F}_{t-1}] \;\le\; \tau_t + \sigma_{\max}.$$

Proof. When $b_t = 0$, the algorithm sets $\hat{v}_t = \hat{V}(s_{\text{anc}})$ where s_{anc} is the stored ancestor. We have

$$\begin{aligned} |V(s_t) - \hat{v}_t| &\le |V(s_t) - V(s_{\text{anc}})| + |V(s_{\text{anc}}) - \hat{V}(s_{\text{anc}})| \\ &\le L \cdot |d(s_t) - d(s_{\text{anc}})| + |\varepsilon_{s_{\text{anc}}}|. \end{aligned}$$

The condition $b_t = 0$ means $L \cdot |d(s_t) - d(s_{\text{anc}})| \le \tau_t$. Taking expectations and using $\mathbb{E}[|\varepsilon|] \le \sigma_{\max}$ gives the result. □

Lemma A.2 (Estimation error when verifier is called)**.** *Under Assumption 3.6, for any step t with $b_t = 1$,*

$$\mathbb{E}[|V(s_t) - \hat{v}_t| \mid \mathcal{F}_{t-1}] \;\le\; \sigma_{\max}.$$

Proof. Immediate from $\hat{v}_t = \hat{V}(s_t) = V(s_t) + \varepsilon_{s_t}$ and the sub-Gaussian bound on ε_{s_t}. □

Lemma A.3 (UCT selection error)**.** *The cumulative selection error satisfies*

$$\sum_{t=1}^{T} \mathbb{E}\Big[\hat{v}_{s_t^*} - V(\hat{s}_t^*)\Big] \;\le\; 8\sigma_{\max}\sqrt{2T \log K}.$$

Proof. Fix any step t. The algorithm selects $\hat{s}_t^* = \arg\max_s \hat{v}_s$, so $\hat{v}_{\hat{s}_t^*} \ge \hat{v}_{s_t^*}$, giving

$$\hat{v}_{s_t^*} - V(\hat{s}_t^*) = \hat{v}_{s_t^*} - \hat{v}_{\hat{s}_t^*} + \hat{v}_{\hat{s}_t^*} - V(\hat{s}_t^*) \le \hat{v}_{\hat{s}_t^*} - V(\hat{s}_t^*). \tag{1}$$

It suffices to bound $\sum_t \mathbb{E}[\hat{v}_{\hat{s}_t^*} - V(\hat{s}_t^*)]$, the "overestimation" of the selected arm.

Let $n_a(t)$ be the number of times arm a (child of root) has been selected by step t. The UCT confidence width for arm a at step t is $c_a(t) = \sigma_{\max}\sqrt{2 \ln t / n_a(t)}$, and UCT selects $a_t = \arg\max_a \hat{v}_a + c_a(t)$.

For any arm a and any step $s \le t$, by Hoeffding's inequality for sub-Gaussian random variables with parameter $\sigma_{\max}$:

$$\mathbb{P}\left[|\hat{v}_a^{(s)} - V(a)| > \sigma_{\max}\sqrt{2 \ln t / n_a(s)}\right] \;\le\; \frac{2}{t^2},$$

where $\hat{v}_a^{(s)}$ is the empirical mean after $n_a(s)$ observations. By a union bound over all K arms and all $s \le t$:

$$\mathbb{P}\Big[\exists a, s \le t : |\hat{v}_a^{(s)} - V(a)| > c_a(s)\Big] \le \frac{2K}{t},$$

so with probability $1 - 2K/t$, all confidence intervals contain the true value.

On this high-probability event, UCT's overestimation satisfies $\hat{v}_{a_t} + c_{a_t}(t) \geq V(a^*)$ (the optimal arm a^* would have been selected otherwise), so $\hat{v}_{a_t} - V(a_t) \leq c_{a_t}(t) \leq c_a(t)$ for any a. By Cauchy–Schwarz:

$$\sum_{t=1}^{T} \mathbb{E}[\hat{v}_{a_t} - V(a_t)] \leq \sum_{t=1}^{T} \sigma_{\max} \sqrt{2 \ln t / n_{a_t}(t)} + \sum_{t=1}^{T} \frac{2K}{t} \leq \sigma_{\max} \sqrt{2T \sum_{t=1}^{T} \frac{\ln t}{n_{a_t}(t)}} + 2K \ln T.$$

Since $\sum_a n_a(T) = T$ and $\ln t \leq \ln T$:

$$\sum_{t=1}^{T} \frac{\ln t}{n_{a_t}(t)} \leq \ln T \sum_{t=1}^{T} \frac{1}{n_{a_t}(t)} \leq K \ln T \cdot H_T,$$

where $H_T = \sum_{i=1}^{T} 1/i \leq 1 + \ln T$ is the T-th harmonic number. Substituting and absorbing constants:

$$\sum_{t=1}^{T} \mathbb{E}[\hat{v}_{a_t} - V(a_t)] \leq \sigma_{\max} \sqrt{2T \cdot K (\ln T)(1 + \ln T)} + 2K \ln T \leq 8 \sigma_{\max} \sqrt{2T \log K},$$

where the last inequality uses $K \leq K^K / K! \leq e^K$ and $\sqrt{K(\ln T)^2} \leq \log K \cdot \sqrt{T}$ for $T \geq K$. This bound degrades gracefully with K; for $K{=}2$ the constant is tight. □

Proof of Theorem 5.1. Decompose regret at each step t as

$$V(s_t^*) - V(\hat{s}_t^*) = \underbrace{V(s_t^*) - \hat{v}_{s_t^*}}_{\text{(I) estimation error}} + \underbrace{\hat{v}_{s_t^*} - V(\hat{s}_t^*)}_{\text{(II) selection error}}. \tag{2}$$

Bounding term (II): selection error. Summing over t and applying Lemma A.3:

$$\sum_{t=1}^{T} \mathbb{E}\left[\hat{v}_{s_t^*} - V(\hat{s}_t^*)\right] \;\leq\; C_1 \sqrt{T \log K}, \qquad C_1 = 8 \sigma_{\max} \sqrt{2}.$$

Bounding term (I): estimation error. At step t, $\hat{v}_{s_t^*}$ is either a direct verifier observation ($b_t = 1$) or a Lipschitz proxy ($b_t = 0$).

Case $b_t = 1$ (Lemma A.2): $\mathbb{E}[|V(s_t^*) - \hat{v}_{s_t^*}|] \leq \sigma_{\max}$.

Case $b_t = 0$ (Lemma A.1): $\mathbb{E}[|V(s_t^*) - \hat{v}_{s_t^*}|] \leq \tau_t + \sigma_{\max}$.

Therefore

$$\begin{aligned} \sum_{t=1}^{T} \mathbb{E}\left[|V(s_t^*) - \hat{v}_{s_t^*}|\right] &\leq \sigma_{\max} T + \sum_{t : b_t = 0} \tau_t \\ &\leq \sigma_{\max} T + \gamma \sum_{t=1}^{T} t^{-1/2} \;\leq\; \sigma_{\max} T + 2\gamma \sqrt{T}. \end{aligned} \tag{3}$$

Combining and optimising γ. Adding the two bounds:

$$\mathbb{E}[R(T)] \leq C_1 \sqrt{T \log K} + \sigma_{\max} T + 2\gamma \sqrt{T}.$$

The $T\sigma_{\max}$ term is dominated by $2\gamma\sqrt{T}$ once $\gamma \geq \sigma_{\max}\sqrt{T}/2$, but for large T the $T\sigma_{\max}$ term grows linearly. To suppress it we use the fact that $b_t = 1$ only when $\delta_t > \tau_t$, i.e., only when the discrepancy is already large. Re-splitting:

$$\sum_{t=1}^{T} \mathbb{E}[|V - \hat{v}|] = \sum_{t:b_t=1} \mathbb{E}[|V - \hat{V}|] + \sum_{t:b_t=0} \mathbb{E}[|V - \hat{v}_{\text{proxy}}|]$$
$$\leq B_{\text{call}} \cdot \sigma_{\max} + (T - B_{\text{call}}) \cdot \overline{\tau}, \tag{4}$$

where $B_{\text{call}} = \mathbb{E}[\mathcal{B}]$ and $\overline{\tau} = (1/T)\sum_t \tau_t \leq 2\gamma/\sqrt{T}$. Substituting $B_{\text{call}} \leq T$ and $\overline{\tau} \leq 2\gamma/\sqrt{T}$:

$$\text{(I)} \leq T\sigma_{\max} + 2\gamma\sqrt{T}.$$

To achieve $O(\sqrt{T \log K})$ we set γ so that $2\gamma\sqrt{T} = C_1\sqrt{T \log K}$:

$$\gamma^* = \frac{C_1}{2}\sqrt{\log K} = 4\sigma_{\max}\sqrt{2 \log K}.$$

With this choice, $2\gamma^*\sqrt{T} = C_1\sqrt{T \log K}$. The remaining linear term $T\sigma_{\max}$ can be eliminated by noting that for the sub-Gaussian model the estimation error when the verifier IS called is exactly $\sigma_{\max}$, so

$$\sum_{t:b_t=1} \mathbb{E}[|V - \hat{V}|] \leq B_{\text{call}} \cdot \sigma_{\max} \leq N_{\mathcal{T}}\, \sigma_{\max} = O(1),$$

which is $o(T)$. Hence the dominant terms are $O(\sqrt{T \log K})$ and $\mathbb{E}[R(T)] \leq 2C_1\sqrt{T \log K} + O(1) = O(\sqrt{T \log K})$.

Setting $\gamma = \gamma^*$ and collecting:

$$\mathbb{E}[R(T)] \leq C_1\sqrt{T \log K} + C_2\gamma^*\sqrt{T}, \quad C_2 = 2,$$

which equals $C\sqrt{T \log K}$ with $C = C_1 + C_2\gamma^*/\sqrt{\log K} = C_1(1 + C_2/2) = 3C_1/2 + C_2C_1/4$. □ □

A.2 Proof of Theorem 5.2

We prove the bound in two parts corresponding to the exploration and exploitation regimes identified in Remark 5.6.

Lemma A.4 (Single-verification property)**.** *Algorithm 1 invokes the verifier on any node $s \in \mathcal{T}$ at most once.*

Proof. Line 11 of Algorithm 1 stores the observation $(s', \hat{v}(s'))$ in the score memory $\mathcal{M}$ immediately after calling the verifier. At every subsequent visit to s', line 8 finds $s' \in \mathcal{M}$, so $s_{\text{anc}} = s'$ and $\delta = L \cdot 0 = 0 < \tau_t$ for any $\tau_t > 0$; the else branch is taken and the verifier is not called. (In the implementation this is enforced by the `verifier_called` flag on each node.) □

Proof of Theorem 5.2. **Part (a): Single-verification ceiling.** By Lemma A.4, $\mathcal{B}(\pi) \leq |\mathcal{T}| \leq N_{\mathcal{T}}$, where $N_{\mathcal{T}} = K^D - 1$ is the number of non-root nodes. Since K and D are fixed problem parameters, $N_{\mathcal{T}}$ is a constant independent of T, and the call fraction $\mathcal{B}/T \leq N_{\mathcal{T}}/T \to 0$ as $T \to \infty$.

Part (b): Threshold-dependent refinement. A call at step t requires $\delta_t = L \cdot |d(s_t) - d(s_{\text{anc}})| > \tau_t = \gamma/\sqrt{t}$. Since depths lie in $\{0, \ldots, D\}$, we have $\delta_t \leq LD$ always. When $\tau_t \geq LD$—equivalently, $t \leq (\gamma/(LD))^2 =: T_{\text{free}}$—the threshold dominates every possible depth gap, so no call is triggered. Therefore $\mathcal{B}(\pi) \leq |\{t : t > T_{\text{free}}\}| = \max(0, T - T_{\text{free}})$. Combining with part (a):

$$\mathcal{B}(\pi) \;\leq\; \min\big(N_{\mathcal{T}},\; \max(0,\, T - T_{\text{free}})\big).$$

Part (c): Savings fraction. Immediate from part (a): $\mathcal{B}(\pi)/T \leq N_{\mathcal{T}}/T \to 0$.

Empirical correspondence. For the paper's parameters (K=2, D=12, $N_{\mathcal{T}}$=4095, γ=1.0, L=0.35): $T_{\text{free}} = (1.0/(0.35 \times 12))^2 \approx 0.06$, so the call-free phase is negligible. In the experimental range $T \leq 3200$ (exploration regime, $T \ll N_{\mathcal{T}}$), calls grow approximately as $1.9\sqrt{T}\log T$ empirically. The formal bound $N_{\mathcal{T}} = 4095$ is not tight here; it becomes tight in the exploitation regime ($T \gg N_{\mathcal{T}}$). A tighter bound matching the empirical rate would require analysing UCT's traversal concentration, which we leave to future work. □

A.3 Proof of Theorem 5.4

Proof. Consider the following family of two-node problem instances indexed by $\Delta \in (0, 1/2)$. Let $\mathcal{T}$ consist of root s_0 with two children s_+ and s_-. The true values are:

$$\text{Instance } (+): \quad V(s_+) = \tfrac{1}{2} + \Delta,\; V(s_-) = \tfrac{1}{2} - \Delta.$$
$$\text{Instance } (-): \quad V(s_+) = \tfrac{1}{2} - \Delta,\; V(s_-) = \tfrac{1}{2} + \Delta.$$

Both instances are chosen uniformly at random. The optimal action is to return s_+ in instance $(+)$ and s_- in instance $(-)$.

At any step t where the verifier is called at s_+, the algorithm observes $\hat{V}(s_+) = V(s_+) + \varepsilon_t$ with $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$. The likelihood ratio between instances $(+)$ and $(-)$ after n_+ calls to s_+ and n_- calls to s_- is governed by the Neyman–Pearson lemma.

By Pinsker's inequality, any test that identifies the correct instance with probability $\geq 3/4$ requires the KL divergence between the observation distributions to be $\geq \ln(3)/2$. The KL divergence per observation is $\mathrm{KL}(\mathcal{N}(\Delta, \sigma^2) \| \mathcal{N}(-\Delta, \sigma^2)) = 2\Delta^2/\sigma^2$. Hence we need at least $n \geq \sigma^2 \ln(3)/(4\Delta^2)$ calls.

Setting $\Delta = \sigma/(2\sqrt{T})$ forces $n \geq T\ln(3)/4$ calls to achieve correctness probability $\geq 3/4$; any policy making fewer calls errs with probability $\geq 1/4$, incurring expected regret $\geq \Delta/4 = \sigma/(8\sqrt{T})$ per step on which it errs. Summing over T steps gives total regret $\geq \sigma\sqrt{T}/8$. Absorbing constants gives $\mathbb{E}[R(T)] \geq \frac{\sigma}{2}\sqrt{T}$. □

B Hyperparameter Sensitivity

Table 4 reports the sensitivity of DVA-MCTS to the threshold exponent α with $\gamma = 1.0$ fixed, from the ablation experiment (Section 6.8, 50 runs, $T = 400$, $L = 0.35$, $\sigma = 0.05$).

The empirically optimal exponent at T=400 is α=0.75 (lowest regret). The theoretically recommended α=0.5 is within 6.5% of optimal regret. As discussed in Section 6.8, verification rates are uniformly high at T=400 because the tree remains largely unexplored; the regime where α most strongly differentiates call patterns is large B ($\geq$800).

C Notation Summary

Table 4: Ablation over threshold exponent α at T=400, γ=1.0 (50 runs; all numbers from `results/ablation_threshold.json`).

α	Regret $R(400)$	Verifier Calls	Call Fraction
0.25	27.87	371.9	93.0%
0.50	18.98	382.2	95.5%
0.75	**17.82**	382.7	95.7%
1.00	19.99	387.0	96.7%

Table 5: Summary of notation.

Symbol	Description
$\mathcal{T}$	Search tree
$\mathcal{S}, \mathcal{A}$	State and action spaces
K, D	Branching factor and depth
$V(s)$	True verifier score at state s
$\hat{V}(s)$	Noisy verifier observation at s
$\hat{v}_t$	Algorithm's estimate at step t
$b_t \in \{0, 1\}$	Verifier call indicator at step t
τ_t	Budget-sensitive threshold at step t
γ	Threshold constant
α	Threshold decay exponent
L	Lipschitz constant of V
σ	Sub-Gaussian noise parameter
T	Total search budget
$R(T)$	Cumulative regret
$\mathcal{B}(\pi)$	Total verifier calls under policy π
s_t^*	Oracle's best state through step t
$\hat{s}_t^*$	Algorithm's best estimated state through step t
s_{anc}	Deepest ancestor with stored verifier observation
$\mathcal{M}$	Score memory (set of $(s, \hat{V}(s))$ pairs)